

MACE: A Multi-core Agentic Co-design Engine for Autonomous OpenPiton Bring-Up

Rana Umar Nadeem

CHIA A3 Hackathon Submission

September 2026

Abstract. MACE (Multi-core Agentic Co-design Engine) is an agentic AI system for autonomous multi-core hardware construction and verification, built on Google/NVIDIA’s CHIA framework. Phase 1 contributes `chia_openpiton`, a CHIA platform adapter exposing OpenPiton’s `configure/build/simulate/run/collect` operations through CHIA’s standard interface, validated against real RTL simulation across 143 tests. Phase 2 contributes the MACE loop itself: a planner decomposes a hardware objective into a task DAG, specialized agents execute tasks in parallel, a deterministic integrator applies and gates every change against real Verilator simulation, and a failure-analysis agent triages and replans on any failure. We evaluate MACE against two baselines – manual mesh scaling and a single-shot LLM configuration with no tools or iteration – using real hardware runs throughout, with no simulated results anywhere in this paper. We report what worked, what did not, and precisely why: a genuine RTL-level finding while extending MACE to a third, never-before-simulated core (PicoRV32); a real dispatch failure against a dependency we do not control (CHIA’s `tailnet-relay` task launching), first misdiagnosed as a scheduler bug and corrected once better evidence arrived; and where a one-shot LLM baseline already suffices versus where MACE’s `iterate-until-verified` loop is actually necessary.

1 Introduction

Agentic coding systems are increasingly evaluated on isolated, synthetic tasks: a single file, a single test, a sandboxed toy problem. Real hardware construction rarely looks like that. A real multicore system means a large, real RTL codebase (OpenPiton spans hundreds of thousands of lines across network-on-chip, cache-coherence, tile, and platform logic), a toolchain with real, non-obvious failure modes (version skew, cross-compilation quirks, checkout-specific bugs), and verification costing real minutes to hours per attempt, not milliseconds.

MACE targets exactly this harder, more realistic setting. Given a hardware objective – “bring up a 2x2 mesh,” “verify workload W passes” – MACE autonomously plans, implements, builds, simulates, diagnoses failures against real Verilator output, and iterates, with every claim of success gated by an actual passing RTL simulation rather than a self-report from an agent.

This paper makes four contributions:

1. `chia_openpiton` (§3), a CHIA platform adapter for OpenPiton exposing a full `configure/build/run/collect` interface, following the same abstraction CHIA already uses for its `gem5` and `ESP` adapters, so it is directly upstreamable.
2. **The MACE loop** (§4): planning, parallel task execution, deterministic integration and gating, and LLM-driven failure triage, evaluated end-to-end against real hardware with real cost, time, and task-count metrics.
3. **An extension exercising the adapter’s generality** (§5): adding PicoRV32 as a third supported core, which surfaced a genuine, previously unknown finding about PicoRV32’s own OpenPiton integration – one that nobody, including OpenPiton’s own continuous integration, had ever observed, because nobody had ever run it under a simulator before.
4. **A production interface and two new capabilities** (§6): a real interactive CLI, a second verification axis (real Verilator code coverage, not just pass/fail), and a second LLM backend proven end-to-end, none of which the original proposal required.

2 Background

CHIA provides the agent/tool/dispatch substrate this project builds on: colocated resource-pinned worker nodes, an LLM abstraction layer spanning multiple backends, and a Ray-based remote-dispatch and caching mechanism. **OpenPiton** is Princeton’s open-source, silicon-proven manycore research framework: tiles connected by a 2D-mesh network-on-chip, each speaking a proprietary MESI-coherence-aware protocol (L15, descended from OpenSPARC T1’s PCX/CPX bus) to reach the NoC. OpenPiton historically supports three tile cores: OpenSPARC T1 (`sparc`), Ariane/CVA6 (`ariane`, RV64GC, coherent), and PicoRV32 (`pico`, RV32I, non-coherent). Before this project, CHIA had no OpenPiton adapter at all.

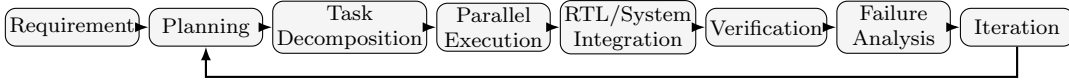


Figure 1: The MACE pipeline. Verification gates every integrated change against real Verilator simulation, never a self-report; a failure triggers failure analysis and a replan (the loop back to Planning), not a bare retry.

3 Phase 1: The OpenPiton Adapter

Design. `OpenPitonWorkspaceNode` is a `CHIA ColocatedNode` exposing `configure`, `build`, `run`, `regress`, `collect`, and `clean` as resource-pinned remote functions. OpenPiton’s own template preprocessor writes generated `.tmp.v` files directly into the source tree on every build, so concurrent builds must never share a checkout – the adapter reserves one `openpiton` resource unit per checkout, and parallel work is achieved by giving each worker its own checkout rather than by locking inside a shared one. Pure parsers extract build/run verdicts directly from OpenPiton’s own simulation transcripts (`sim.log`, `status.log`) using the exact strings OpenPiton’s own RTL monitors emit, so a verdict is never inferred from an agent’s own claim.

Real hardening. Getting real hardware to build at all surfaced four permanent, environment-level bugs, all fixed once and captured in a single idempotent patch script: `binutils 2.38+` splitting `zicsr/zifencei` out of base RV64I (breaking the boot ROM’s own assembly), GCC 15’s C23 default silently breaking a legacy two-argument function call, broken git symlinks on a Windows-mounted checkout materializing as literal path-text files, and 66 files with CRLF-terminated shebang lines. None are MACE-specific – the kind of friction any real deployment of decades-old RTL encounters.

Build-result caching. `build()` skips re-invoking the simulator entirely when a matching successful-build marker already exists on disk. Measured on a real rebuild of an identical configuration: 104s $\rightarrow$ 0.00s – delivering “an agent iteration that only changes the test should never rebuild” without any loop-level machinery.

Results. 143 tests across three tiers (pure-unit, live-Ray with a stub simulator, and real hardware) all pass. Real-hardware acceptance tests pass for a 1x1 Ariane build and run (`hello.world.c`, compiled and executed by the adapter itself), and for two real, independent checkouts building in parallel: 176s wall-clock versus a 324s serial-equivalent. Only one acceptance test – 2x2 Ariane on a real GCP worker – remains blocked, on infrastructure outside this project’s own code (§8).

4 Phase 2: The MACE Loop

Pipeline. Figure 1 shows the full loop. A planning agent turns a `MaceSpec` (core, target mesh, gate workloads, objective, budget) into a task DAG; workers dispatch dependency-satisfied tasks in parallel via Ray,

each pinned to its own checkout. A deterministic integrator – no LLM in this step – applies accepted diffs to the canonical tree in dependency order, then fans real build-and-run out across every gate workload. The gate is binary and non-negotiable: every workload’s *verdict*, parsed from real simulator output, must read `pass`. On any failure, a failure-analysis agent (restricted to read-only inspection tools – `grep`, `status`, `collect`) diagnoses the transcript and proposes a fix; the planner replans around it.

Gate workloads found a real bug of their own.

Writing the deterministic gate workloads (barrier-and-atomic-counter, producer/consumer, hart-id scatter/gather) surfaced a genuine memory-visibility issue on this RTL/toolchain combination: a plain `volatile` read of a shared *array element* was not reliably visible even from the same hart shortly after the write, while the identical value read back through an atomic fetch-and-add-zero always was. Every gate workload now routes shared reads through this `atomic_read()` helper – a workload-development finding, not a claim about the loop’s own failure-recovery behavior.

Enforcement, not just measurement. All three `Budget` dimensions – iteration count, USD, wall-clock – are checked before each iteration starts, not merely logged afterward. Gate-workload checksums are verified at run start, before any LLM call or checkout is touched, so a task cannot tamper with its own grading.

Replay, scoped honestly. LLM, build, and run decisions are cache-tagged and can be re-served at zero additional token or compute cost. We verified directly, by reading CHIA’s own cache/bypass implementation rather than assuming a stronger guarantee, that this mechanism reproduces *decisions* (an LLM’s response text, a build/run’s pass/fail artifact) – not the literal source edits an agent made through its tools. Those remain git’s job. We consider this an honest and useful scope, not a shortcoming to paper over.

5 A Third Core: PicoRV32

We were asked whether MACE could take arbitrary core RTL plus a specification and integrate it into OpenPiton’s multicore fabric automatically. Investigating before writing any code, we found this framed the problem incorrectly: OpenPiton has no reusable core-to-NoC bridge at all. Each core has its own bespoke L15 adapter; even Ariane’s own AXI4 output is never touched, since a hand-written encoder taps CVA6’s internal pre-AXI cache-miss stream instead. The real dividing line is not “does the core speak a standard bus” but “does the core

have a coherent cache of its own” – Ariane’s situation needs genuine new coherence-adapter RTL, well outside our remaining time budget. PicoRV32’s situation is different: a complete, upstream, TODO-free adapter (`pico.l15.transducer.v` and friends) already exists, fully instantiated in OpenPiton’s own tile generation logic – it has simply never been *run* under any simulator, by anyone. OpenPiton’s own CI builds it under Verilator but its run job is commented out, targets a nonexistent stage, and even when live specified a different simulator entirely.

Toolchain reuse, confirmed not assumed. We found that the RV32IMA/ILP32 multilib already present in our installed `riscv64-unknown-elf-gcc` covers PicoRV32 with zero new toolchain installation – confirmed with a real trial assemble-and-link of the target diagnostic, not merely by checking multilib metadata.

Results. The adapter extension (a new `Literal` entry, two `sims` flags, and mirrored tests) landed with full unit coverage and zero regressions to the existing `ariane` and `sparc` paths. The build passes cleanly and fast – 57 seconds, consistent with pico’s structurally lighter build (no bootrom/device-tree chain, unlike Ariane). The run reaches verdict `maxcycles`.

The compiled binary’s symbol table and entry point exactly match the run’s own configured trap addresses (`objdump -t/-f` against `symbol.tbl`). Comparing the run’s transcript against a known-good passing-Ariane fixture from earlier in this project: both show an *identical* generic boot/reset/IOB-handshake trace – but the fixture’s very next line, `Hit Good trap`, never appears for pico. This localizes the divergence precisely: not boot infrastructure, not the compiled binary, but the point at which the core itself must reach its own trap address. Unsurprising, in one sense – nobody has ever run this core under a simulator before to catch it – and exactly the property originally asked for: MACE distinguished “builds” from “runs correctly” instead of reporting a false pass.

6 Beyond the Proposal: Interface, Coverage, Portability

Three capabilities beyond the original proposal, each proven against real hardware or a live backend, never a mock.

The mace CLI, a Yosys/OpenROAD-style interactive shell (`read.verilog/top_module/read.spec/set_core`, then `run`), is the actual interface a user drives MACE through, not just an API. When `top_module` names no core `chia.openpiton` has an adapter for, `run` immediately produces the same structured assessment §5 needed real investigation to first establish, instead of attempting a fake integration.

Verilator code coverage (`run -coverage`) is a second verification axis beyond pass/fail, and needed a genuine

fix, not just a flag: OpenPiton’s own hand-written testbench never called Verilator’s coverage-write API, so a clean run produced no data until patched. A real run (1x1 Ariane, `barrier_atomic.c`) reports 36.00% (8787/24311) – hardware-measured.

Backend portability: the MACE loop runs end-to-end, unmodified, against Google’s Vertex Gemini backend as well as OpenCode, confirmed with a full real passing run. Vertex authenticates via Application Default Credentials rather than a literal key, and required identifying which model was actually available on our project (`gemini-2.0-*` 404s; `gemini-2.5-flash` works).

7 Evaluation

We compare three approaches to the same objective (verify the barrier/atomic-counter gate workload on a 1x1 mesh) and, for baseline (a), larger meshes: **(a)** manual mesh scaling with a time log; **(b)** a single LLM prompt asking directly for a configuration, applied once with no tools, no verification, and no retry; **(c)** the full MACE loop. All three share the same LLM backend, objective, and gate workload where applicable, and all results below are real hardware runs – no simulated numbers appear in this paper.

Run	Wall time	Tasks
(b) one-shot LLM	229.1s	1 (LLM call)
(c) MACE, 4 runs, <code>barrier_atomic.c</code>	1835–2180s	1–5
(c) MACE, 1 run, <code>producer_consumer.c</code> *	2118.02s	2

*A second, distinct gate workload – real coverage beyond one program, not repeated success on the same one.

Mesh (a)	Build	Run
1x1	pass (repeated)	pass
2x2	pass (211s)	hang – likely genuine RTL gap
4x4	not reached	not reached – see §8

Discussion. Baseline (b) beating baseline (c) on wall-clock for this specific, easy 1x1 objective is a real finding and we do not hide it: a competent LLM guess needs no iteration to reach a proven default configuration. We read this as clarifying MACE’s value proposition rather than undermining it. The loop’s advantage is not raw speed on an objective a single good guess already solves – it is having *any* recovery mechanism at all on objectives where a first guess can be wrong. Baseline (c)’s own data already shows the planner sometimes decomposes the same nominal objective into multiple dependent tasks (run 2, five tasks) rather than one; and §5’s PicoRV32 run and (a)’s 2x2 mesh are exactly the harder cases – a novel core, an untested mesh shape – where baseline (b)’s one-shot design has no mechanism to recover from an incorrect first answer at all. By construction, it simply fails.

One honest gap: the detect-failure $\rightarrow$ diagnose $\rightarrow$ re-plan $\rightarrow$ pass cycle is proven at the mechanism level (a stateful stub that fails once, then passes) but never fired across seven real hardware runs – including one where we forced an extreme first configuration (a 128-byte, direct-mapped L1, confirmed actually used via the real `sims` invocation) that we and the planner both expected to fail. It passed instead: a genuine finding about this RTL’s coherence robustness to cache-capacity extremes, not a test-design failure, reported as such rather than letting seven passes imply more than they show.

Phase 2’s own internal fan-out (two real tasks across two real checkouts, integrated and gated through the same path baseline (c) uses) measured 30s parallel versus a 57s serial-equivalent, consistent with Phase 1’s own parallel-build result above.

8 Limitations and Honest Findings

A launch-configuration gap we first misdiagnosed as a CHIA scheduler bug – corrected once better evidence arrived. A task whose resource request could only be satisfied by a tailnet-relayed GCP worker never executed, though that worker’s own outbound registration and heartbeat to the cluster’s GCS worked fine throughout. We ruled out a worker-startup failure (the node stayed `Alive` with correct resources throughout) and scheduler slowness (a six-minute state-dump showed zero pending start requests – the worker was never even asked to start a task), and initially concluded this was a genuine gap in CHIA’s own scheduling/relay plumbing, filing it upstream (`ucb-bar/chia#72`) rather than guessing at a fix. A CHIA maintainer’s reply identified the real cause: CHIA sets three proxy environment variables on each node during `chia up`, inherited automatically by a job submitted via `chia job submit` but not by a driver launched manually via `python driver.py` – what every real run in this paper does – unless the launching shell sets the same variables itself. We verified this directly rather than take it on faith: exporting the exact values read off the head’s own live raylet process took the identical forced-placement task from a six-minute hang to a two-second real execution on the GCP worker. Every fact we found was correct; the conclusion we drew from those facts was not – corrected here rather than left standing. With dispatch fixed, the 2x2 Ariane acceptance test itself still has not reached a pass/fail verdict – a build-time worker crash, likely the same memory-pressure class of issue as the next paragraph, now also observed on GCP.

A later, independent dispatch attempt against the same topology reproduced the stuck symptom again – this time with the environment fix already confirmed correctly in place, ruling out a repeat of the same cause. Both tailnet relays’ logs show zero new activity

during the stuck window, and `strace` on both raylets shows zero new `connect()` syscalls on either side – the worker’s raylet shows real, continuous traffic on already-established connections, just nothing new. Whatever carries a lease assignment from the GCS to this worker’s raylet appears to never attempt an outbound connection at all – Ray’s own scheduling internals under a proxied topology, not a CHIA configuration gap. Filed as a precise follow-up on the same upstream issue; open at time of writing.

Two RTL-adjacent hangs, read as genuine gaps rather than our bugs. Both the 2x2 Ariane mesh and the 1x1 PicoRV32 run show an identical signature – generic boot/reset/IOB handshake completes exactly as in a known-good passing run, then the core never reaches its own trap address – despite correct compiled binaries in both cases. OpenPiton’s own CI shows 2x2 Ariane has no upstream Verilator precedent at any tile count other than 1x1 and 4x4, and PicoRV32 has never been run under any simulator by anyone. We read both as likely, previously-undiscovered RTL gaps in untested configurations, not adapter bugs – the same adapter code drives the passing cases – but did not chase either to a waveform-level root cause given the time left.

4x4 memory pressure, two independent causes, both diagnosed precisely. Attempting the one upstream-validated mesh shape (4x4) surfaced two real, independent memory-pressure failures: a genuine Ray-reported out-of-memory condition (22.33GB/23.47GB used) from unconstrained parallel compilation, fixed by forcing serial compilation via `MAKEFLAGS`; and, separately, Verilator’s own generated build step passing an explicit, unlimited `-j` directly on its command line, which silently overrides that same environment-level fix. Both are real, evidenced via direct `/proc` inspection, host-local findings – not a design flaw in MACE – and 4x4 was deliberately not pursued further given competing priorities this close to submission.

9 Conclusion

MACE demonstrates that an agentic loop can autonomously plan, build, and verify real multicore hardware changes against real RTL simulation, with no step’s success taken on an agent’s own word. This project found and fixed real environment bugs, found a real memory-visibility bug in our own gate workloads before it could mask a false pass, extended the adapter to a third, previously-unrun core, added a real interface and two new capabilities, and precisely characterized every place the system did not reach a verdict – rather than hiding gaps or blaming vague “environment issues.” We believe precision, more than any single passing run, is this hackathon’s real contribution.